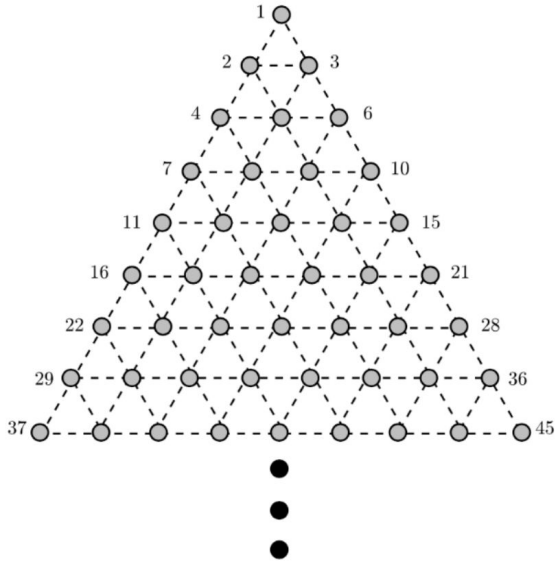
# PROYECTO 03 LENGUAJES DE PROGRAMACIÓN
# FIGURAS GEOMETRICAS

Andrés Ottón Brenes Maleaño
brends31@estudiantec.cr
Axel Alejandro Fernández Jiménez
axl1210@estudiantec.cr

## 1. PROBLEMA A RESOLVER

Se debe dictar si a partir de vértices brindados, se puede formar una figura geométrica. Estos vértices serán brindados por el archivo que se debe abrir y leer, y el sistema debe definir si los vértices forman una figura geométrica, entre las figuras que se pueden crear en este sistema se encuentran: triángulos, hexágonos, paralelogramos. El límite de vértices es de 32767, pues así fue definido, dándonos la oportunidad de crear figuras geométricas desde 1 hasta ese número dictado.




## 2. REQUERIMIENTOS

Para correr el proyecto es necesario poseer un compilador de Prolog, el código fuente y un archivo de texto con los números de los vértices que se desean evaluar. Una vez con estos archivos, se inserta en el compilador de prolog el código fuente y se inicializa el programa de la manera: “main…”

## 3. IMPLEMENTACIÓN PROPUESTA

El problema fue resuelto de una manera matemática, desde el momento en que el triángulo se forma de una suma de i hasta n, donde i es igual a 1 y n es 32767, esto nos permite de cierta manera interactuar con los datos solo con la cabeza y la cola de cada profundidad presente en el triángulo. Debido a que los números pueden leerse en un orden aleatorio y el usuario desconoce que deben estar en un orden de menor a mayor, por lo que se programó un quicksort para ordenar los elementos leídos en el archivo.

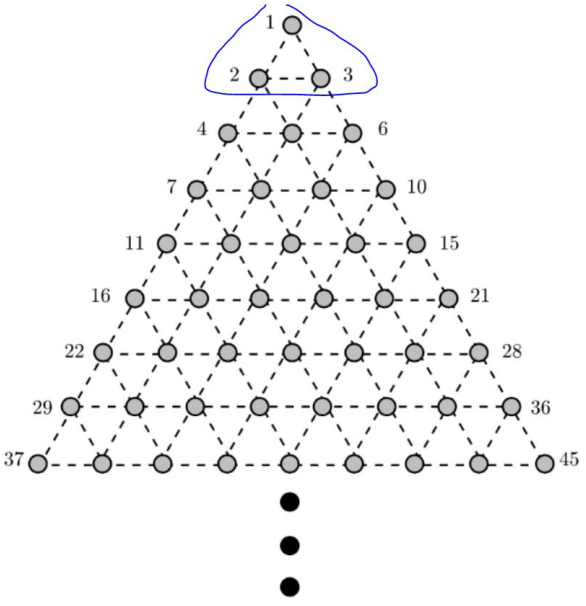
## 4. RESULTADOS

La salida del archivo debe dictar si los vértices leídos y analizados forman una figura geométrica en el triángulo creado. Las tres posibles figuras geométricas posibles son: triángulo, paralelogramo y hexágono.

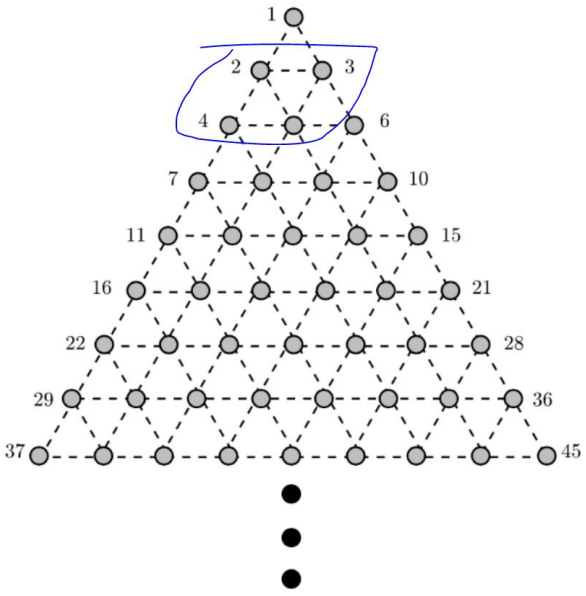
Para crear las excepciones se crearon funciones específicas que verifiquen los vértices ingresados sean en cada caso específico de la siguiente manera: el triángulo puede existir, si o solo si se leen 3 vértices. Así como el paralelogramo puede existir solo si se leen 4 vértices, y por último el hexágono se representa si y sólo si se leen 6 vértices.

La distancia entre los vértices debe ser la misma en todos los casos, por lo que si un paralelogramo posee de distancia n entre los primeros vértices y entre los primeros vértices y los segundos vértices existe una distancia m, esto no será considerado un paralelogramo.

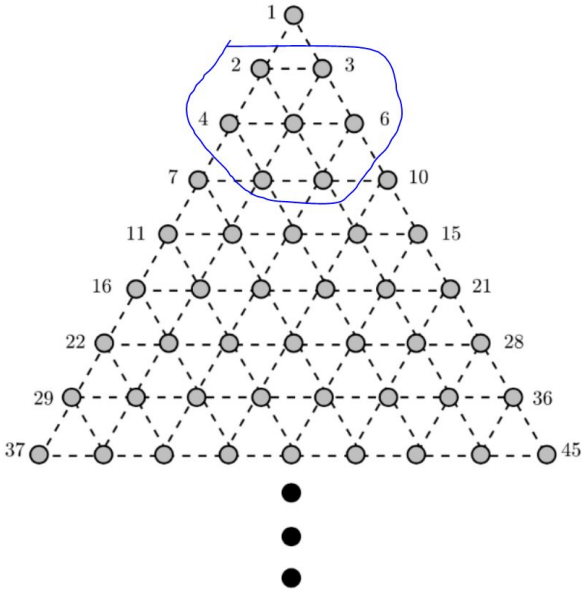
Ejemplo de un triángulo {1,2,3}:




Ejemplo de un paralelogramo {2,3,4,5}:




Ejemplo de un hexágono {2,3,4,6,8,9}:




## 5. EXPERIENCIAS

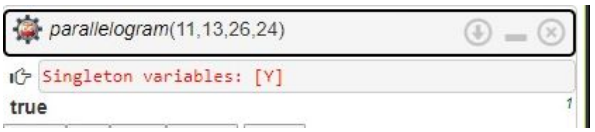
Al ser un lenguaje de paradigma lógico, la abstracción es más difícil y por lo tanto pensar en la solución correcta al programa se hace muy complicado. Este lenguaje es muy fuerte para sistemas de inteligencia artificial, por su énfasis en los predicados.

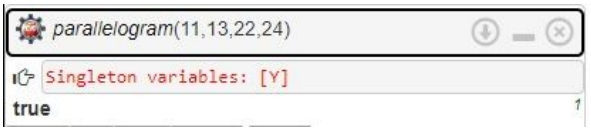
El desarrollo de este proyecto se vio afectado por la poca capacidad que se tuvo de abstraer los resultados que se daban a los distintos problemas o consultas.

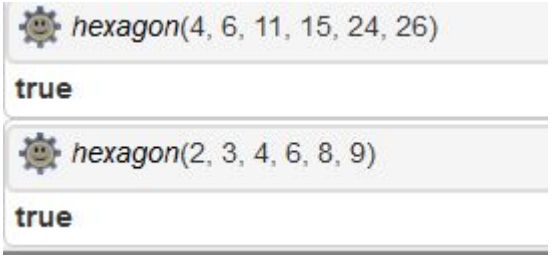
## 6. PRUEBAS

```
?- triangle(11,22, 24).
true .

?- triangle(11, 12, 17).
true .
```




## 7. AMBIENTE DE DESARROLLO UTILIZADO

El ambiente de desarrollo utilizado en el proyecto fue SWI Prolog. Este es solo el compilador por lo que el código debe ser desarrollado en un editor de texto fuera de SWI Prolog. Este compilador nos da la oportunidad de ver los resultados del código.




El código fue escrito en el editor de texto Sublime 3.



# 8. REFERENCIAS BIBLIOGRÁFICAS


- Swi-Prolog (s.f). Reference Manual. Recuperado de: http://www.swi-prolog.org/pldoc/doc_for?object=manual
- Sublime HQ Pty Ltd (s.f) Sublime Text. Recuperado de: https://www.sublimetext.com
- Wikipedia(s.f) Sumatorio. Recuperado de: https://es.wikipedia.org/wiki/Sumatorio